

Additional References

- Primarily the OpenAI Agents SDK: official documentation, source and examples, the Agents API, and GPT-Live
- OpenAI platform guides appear only where a course demo uses them (hosted tools, MCP, conversation state, models)
- Recent articles (May–September 2026) at the end cover topics that go beyond the course notebooks

Agents SDK Documentation — Core

https://openai.github.io/openai-agents-python/

- Agents SDK documentation home — official docs for agents, runners, tools, handoffs, guardrails, sessions, MCP, sandbox, realtime and voice agents, and tracing.
- Quickstart — install the SDK, set the API key, and build a first agent with handoffs and guardrails.
- Configuration — SDK-wide defaults: API key and client, Responses vs. Chat Completions API shape, tracing export, and logging.
- Agents — instructions, dynamic instructions, context, output types, model settings, hooks, cloning, and forcing tool use.
- Running agents — `Runner.run`, `Runner.run_sync`, `Runner.run_streamed`, run config, error handlers, and durable-execution integrations.

- Results — final output, new items, raw responses, interruptions and run state, guardrail results, and usage.
- Streaming — raw response events, run item and agent events, streaming with approvals, and cancellation.
- Tools — hosted tools, local runtime tools (`ComputerTool`, `ShellTool`, `ApplyPatchTool`), function tools, agents-as-tools, tool search, and approval gates.
- Guardrails — input, output, and tool guardrails for policy and topic checks.
- Handoffs — routing work between specialized agents, handoff inputs, and input filters.
- Agent orchestration — orchestrating via the LLM (handoffs, agents-as-tools) versus orchestrating in code.
- Context management — local context (`RunContextWrapper`, `ToolContext`) versus context the model can see.
- Human-in-the-loop — marking tools as needing approval, interruptions, and resuming runs from saved state.
- Usage — per-run token usage from the run context for cost monitoring and limits.
- Tracing — traces and spans, custom trace processors, and third-party observability integrations.
- Testing — deterministic in-memory test utilities for agents, sandbox, realtime, and voice without model calls.
- Agent visualization — Graphviz diagrams of agents, tools, handoffs, and MCP servers (`draw_graph`).
- REPL utility — `run_demo_loop` for interactive terminal testing with persistent history and streaming.
- Release process and changelog — 0.Y.Z versioning, breaking-change policy, and the changelog.
- API reference — class-by-class reference, including extensions such as Redis/MongoDB sessions, compaction sessions, and the Codex tool.

Agents SDK Documentation — Sessions, Models, and MCP

- Sessions — conversation history with `SQLiteSession`, OpenAI Conversations sessions, Redis, MongoDB, SQLAlchemy, Dapr, and custom implementations.
- SQLAlchemy session — production session storage on any SQLAlchemy-supported database (PostgreSQL, MySQL, SQLite).
- Advanced SQLite session — SQLite session with conversation branching, per-turn usage analytics, and structured queries.
- Encrypted session — transparent encryption wrapper for any session, with automatic expiration of old items.
- Models — choosing and configuring models, mixing models, retries and timeouts, and non-OpenAI providers; the default model is `gpt-5.6-luna`.
- Third-party adapters (LiteLLM, Any-LLM) — `LitellmModel` and other adapters for 100+ providers.
- Model context protocol (MCP) — hosted MCP tools, `MCPServerStreamableHttp`, `MCPServerStdio`, tool filtering, and server management.

Agents SDK Documentation — Sandbox, Realtime, and Voice Agents

- Sandbox agents quickstart — `SandboxAgent` runs with a filesystem workspace, shell access, memory, compaction, and skills.
- Sandbox concepts — workspaces, capabilities, tools, and how a sandbox run proceeds.
- Sandbox clients — where sandbox work runs: local, Docker, or hosted providers via `SandboxRunConfig`.
- Sandbox agent memory — distilling lessons from prior runs into workspace files the agent reads next time.
- Realtime agents quickstart — `RealtimeAgent` and `RealtimeRunner` on the Realtime API with `gpt-realtime-2.1`.
- Realtime agents guide — how the SDK's realtime layer maps onto the Realtime API and what the Python SDK adds.
- Realtime transport — server-side WebSocket versus SIP transport for realtime agents.
- Voice pipeline quickstart — speech-to-text, agent, and text-to-speech pipeline entry point.
- Voice pipeline guide — pipeline configuration, audio input types, and result streaming.
- Voice pipeline tracing — automatic tracing of voice pipelines and configuring it via `VoicePipelineConfig`.

Agents SDK Source, Examples, and the TypeScript SDK

- Agents SDK GitHub repository — source code, issues, and releases for the Python SDK.
- Agents SDK release notes — what each release adds or breaks; the course was verified against 0.22.x.
- Examples index — official examples page that links to the GitHub examples directory.
- Examples source tree — `basic`, `tools`, `agent_patterns`, `handoffs`, `memory`, `mcp`, `hosted_mcp`, `research_bot`, `financial_research_agent`, `model_providers`, `realtime`, `voice`, `sandbox`.
- Research bot example — planner, parallel search agents, and writer multi-agent research workflow.
- Financial research agent example — planner, search, analyst tools, writer, and verifier agents for finance use cases.
- Sandbox examples — sandboxed-agent examples: basics, handoffs, memory, cloud backends (E2B, Modal, Daytona), and guided tutorials.
- Agents SDK for TypeScript — the TypeScript SDK's guides: agents, sandbox agents, sessions, voice agents, tracing, and testing.
- Agents SDK for TypeScript repository — source and releases for the TypeScript SDK, which tracks the Python SDK feature by feature.

Related OpenAI Platform Guides

- Agents guide hub — OpenAI's comparison of the Responses API, the Agents SDK, and the Agents API, with links to each.

- Agents SDK on the platform docs — platform-side overview of the SDK with pages on defining agents, models, running agents, sandboxes, orchestration, guardrails and approvals, results, and observability integrations.
- Models — current catalog: GPT-6 Astra, the GPT-5.6 Sol/Terra/Luna tiers, GPT-Live 1, GPT-Realtime 2.1, and GPT Image 2.5.
- Reasoning models — reasoning effort settings used in `ModelSettings(reasoning={'effort': ...})`.
- Conversation state — `previous_response_id`, the Conversations API, context-window management, and compaction.
- Conversations API reference — the server-managed conversation objects behind `conversation_id`.
- Web search tool — hosted web search, agentic search, domain filtering, and citations.
- File search tool — vector stores, metadata filtering, and retrieval-augmented generation.
- Code Interpreter tool — running Python in hosted containers and retrieving generated files.
- Computer use — model-driven browser or desktop control via code execution or the `computer` tool, plus safety controls.
- Image generation tool — image generation as a model tool: `gpt-image-2.5` models, multi-turn editing, and streaming.
- Image generation guide — `gpt-image-2.5-sunburst` versus `gpt-image-2.5-flare`, quality levels, sizes, and editing.
- Connectors and MCP servers — remote MCP servers and OpenAI connectors as tools, with approvals, OAuth, and tool filtering.
- Skills — packaging reusable instructions and scripts as skills that `SandboxAgent`s load on demand.
- Programmatic tool calling — letting the model write code that coordinates several tool calls in one step.

OpenAI Agents API (Public Beta, September 10, 2026)

OpenAI runs the agent loop for you on the Codex harness. Reached through the OpenAI Python SDK (`client.beta.agents`), not the Agents SDK.

- Introducing the Agents API — announcement: managed sessions, context compaction, tool search, programmatic tool calling, subagents, and hosted or partner sandboxes.
- Agents API overview — agents, environments, sessions, and events; how it compares to the Agents SDK and the Responses API.
- Agents API quickstart — `client.beta.agents.sessions.create(...)` with an OpenAI-hosted sandbox and streamed events.
- Architecture — how the harness, sessions, sandboxes, and tools fit together.
- Sessions — durable sessions: creating, steering, continuing, and managing them, plus events and webhooks.
- Function tools — application-side function handlers the managed agent can call.
- MCP tools — connecting managed agents to MCP servers.

- OpenAI-hosted environments — sandboxes OpenAI provisions for running code and working with files.
- Self-hosted environments — running the agent's work on your own infrastructure or a partner sandbox provider.
- Multi-agent — subagents with independent context for parallel, delegated work.
- Observability and tracing — monitoring managed sessions and exporting traces.
- Agents API examples (OpenAI Cookbook) — five sample apps (incident response, Slack bot, data analyst, GitHub investigator, doc reviewer) and self-hosted sandbox patterns; September 10, 2026.

GPT-Live and Voice

- Getting started with GPT-Live — full-duplex voice conversations that delegate work to a backend agent.
- Delegation and tools in GPT-Live — Responses versus client delegation, `session.thinking` and `session.commentary` updates, and returning results.
- Prompting GPT-Live — conversation prompts with backchannel, interruption, and delegation policies.
- Managing GPT-Live sessions — session lifecycle, transcripts, usage events, and graceful close.
- Migrate to GPT-Live — adding a GPT-Live voice front end to an existing text agent.
- GPT-Live over WebSockets — the client and server events `source/live_voice.py` sends and handles.
- Live API reference — endpoint and event reference for `client.live`.
- GPT-Live 1 model — capabilities, $0.05 per voice minute billed per second, and rate limits.
- Voice agents guide — choosing between GPT-Live, the Realtime API, and a speech-to-text/text-to-speech pipeline.
- Realtime API guide — the API behind the SDK's `RealtimeAgent`, with `gpt-realtime-2.1`; positions GPT-Live as the full-duplex alternative.
- GPT-Live evaluation guide (OpenAI Cookbook) — crawl/walk/run eval harnesses for full-duplex voice agents that check delegated tool execution and application state; September 10, 2026.
- Building a Voice Assistant with the Agents SDK (OpenAI Cookbook) — the SDK voice-pipeline example (speech-to-text, agent, text-to-speech); March 2025 but still the official pipeline walkthrough.

Local Models and Model Providers

- LiteLLM documentation — routing OpenAI-style calls to many model providers; current releases require `openai` 2.x, hence the separate `deitel-openai-litellm` environment.
- LiteLLM Ollama provider — the `ollama/` model prefix and base URL settings used with `LitellmModel`.
- Ollama documentation — official docs for the local model runtime: installing, pulling models, and serving.

- Ollama OpenAI compatibility — Ollama's OpenAI-compatible endpoint, an alternative path via the SDK's `OpenAIChatCompletionsModel` without LiteLLM.
- How to run gpt-oss locally with Ollama (OpenAI Cookbook) — running OpenAI's open-weight models locally, including an Agents SDK integration.

MCP Ecosystem

- Model Context Protocol documentation — protocol concepts, architecture, and guides for building servers and clients.
- MCP Python SDK — official Python SDK (v2) for building MCP servers and clients.
- Awesome MCP Servers — curated community list of MCP servers.
- MCPServers.com — community directory for discovering MCP servers.
- mcpservers.org — additional directory, including remote servers.
- Microsoft Playwright MCP — reputable example of an MCP server for browser automation.

Libraries and Background Used by the Demos

- Playwright for Python — browser automation library.
- playwright-python repository — source and issue tracker for the Python Playwright package.
- yfinance documentation — simple financial market-data package.
- SQLite documentation — the database behind `SQLiteSession`.
- Redis documentation — background for the SDK's Redis session backend.
- MongoDB PyMongo documentation — background for the SDK's MongoDB session backend.
- Agent Skills standard — the open `SKILL.md` standard adopted by Codex, Claude, Cursor, Copilot, and the Agents SDK's sandbox agents.
- Building Effective AI Agents (Anthropic) — the workflow taxonomy (routing, parallelization, orchestrator-workers, evaluator-optimizer) that the SDK's `agent_patterns` examples implement.

Recent Articles on the OpenAI Agents SDK

Selected for topics that go **beyond** the course notebooks: sandbox agents, durable execution, production sessions, evals and observability, deployment, MCP gateways, and the Agents API

From OpenAI

- Introducing the Agents API — OpenAI, September 10, 2026. Public-beta managed Codex harness: sessions, compaction, tool search, programmatic tool calling, subagents, and hosted or partner sandboxes.
- Agents API examples — OpenAI Cookbook, September 10, 2026. Five Agents API apps (incident response, Slack bot, data analyst, GitHub investigator, doc reviewer) plus app-managed and webhook-managed self-hosted sandbox patterns.

- GPT-Live evaluation guide — OpenAI Cookbook, September 10, 2026. Crawl/walk/run eval harnesses for full-duplex GPT-Live voice agents that check delegated tool execution and application state, not just transcripts.
- Build a ChatGPT plugin with the OpenAI Agents SDK and Amazon Bedrock AgentCore — Rohan Awasthi, Chuck Hernandez, Syed Ahmed, Steven McAteer (OpenAI Cookbook), September 3, 2026. Deploys an SDK agent behind a private MCP adapter via Secure MCP Tunnel, optionally on AgentCore Runtime, with dual tracing to CloudWatch and OpenAI.
- Combining security scanners with the Agents SDK — OpenAI Cookbook, August 5, 2026. A security-review swarm using `Agent.as_tool()`, `tool_use_behavior` stop functions, `is_enabled` tools, approval gates, and provenance validation.
- Enabling Long-Term Agent Memory with Oracle AI Agent Memory — OpenAI Cookbook, Oracle partner entry, July 16, 2026. Implements the SDK `Session` protocol on Oracle AI Database and separates durable long-term memory tools from session replay.

OpenAI Developer Blog

https://developers.openai.com/blog — recent posts relevant to agents, skills, and MCP.

- Rethinking skills and prompts for GPT-6 Astra — OpenAI, September 11, 2026. Trim skill descriptions, `AGENTS.md` files, and task prompts for a more capable model; over-specified instructions now cost accuracy and context.
- Codex as a platform: build on the open agent harness — OpenAI, August 19, 2026. Embedding the open-source Codex harness (the loop the Agents API now hosts) in your own products via `codex exec`, the Codex SDK, and app-server.
- Making private MCP servers reachable without making them public — OpenAI, June 26, 2026. Secure MCP Tunnel: an outbound-only client that lets OpenAI products call MCP servers inside a private network, with OAuth through the tunnel.

Workspace Agents

- No-code AI agents that run inside ChatGPT workspaces
- Designed for organizations to automate recurring business workflows
  - Access approved enterprise data and connectors
  - Execute multi-step tasks autonomously
  - Run scheduled workflows
  - Share agents across an organization
- Intended for business users rather than application developers
- Managed by workspace administrators
- Recommended by OpenAI for natural-language workflow automation instead of code-based agent development

Availability

- ChatGPT Business
- ChatGPT Enterprise
- ChatGPT Edu
- ChatGPT Teachers (where available)

References

- Introducing workspace agents in ChatGPT — OpenAI's announcement of shared, Codex-powered workspace agents in ChatGPT and Slack.
- Workspace Agents (OpenAI Help Center) — availability, permissions, and administration.

© 2026 by Deitel & Associates, Inc. All Rights Reserved.